
Long-Context QA Dataset for Post-Training LLMs

Cuveillier Nicolas

IC School, EPFL
nicolas.cuveillier@epfl.ch

Kolly Florian

Neuro-X Institute, EPFL
florian.kolly@epfl.ch

Stojanovic Marko

School of Engineering, EPFL
marko.stojanovic@epfl.ch

Wohlfahrt Oscar

School of Basic Sciences, EPFL
oscar.wohlfahrt@epfl.ch

Abstract

Large Language Models (LLMs) frequently bypass reasoning from context in favor of memorized knowledge or lexical matching, undermining their reliability in Long-Context tasks. Reinforcement learning (RL) post-training on question-answer (QA) datasets has emerged as a promising approach to provide more faithful reasoning. Yet, existing benchmarks are either too limited in scale or consist of single-hop questions easily answerable.

Here, we propose an automated pipeline that generates large-scale, multi-hop QA datasets from large raw document without the need for human annotation. Our pipeline extracts structured relational triples, verifies their textual support via Natural Language Inference (NLI), chains verified triples into multi-hop reasoning paths, and synthesizes questions that require integrating evidence across multiple text passages. We instantiate this pipeline on the Institutional Books corpus and use the resulting datasets to post-train Qwen2.5-7B-Instruct under multiple RL policies and reward functions.

Our results demonstrate some improvements in long-context document-grounded reasoning. These initial experiments confirm that even state-of-the-art models still originally lack the ability to answer our multi-hop difficult questions, but are able to improve during post-training.

1 Introduction

The ability to reason faithfully over provided documents is a prerequisite for deploying Large Language Models (LLMs) in knowledge-intensive applications such as legal analysis, scientific literature review, and business reports. Long-context inference refers to the task where the model processes an entire document directly within its context window rather than retrieving relevant passages from an external index as in Retrieval-Augmented Generation (RAG) [6]. Yet, recent work has highlighted a fundamental limitation: models frequently ignore the context as it grows, defaulting instead to parametric knowledge acquired during pretraining [7].

A natural approach to correcting this failure mode is post-training. Here, reinforcement learning offers a powerful method: by rewarding the model only when it produces a verifiably correct answer, RL post-training creates a direct signal tied to document comprehension rather than pattern matching. This paradigm has proven effective in mathematical reasoning [8], where rule-based reward functions provide unambiguous feedback, and there is growing interest in extending it to open-domain question answering [21].

The effectiveness of RL post-training is, however, critically dependent on the quality of the underlying QA dataset. Three properties are essential. First, questions must be *unanswerable from memory alone*,

forcing the model to consult the document. Second, they must require *multi-hop reasoning*, integrating evidence from multiple passages rather than matching a single retrievable span. Third, answers must be *automatically verifiable*, enabling scalable reward computation without human judges. Existing datasets satisfy at most one or two of these criteria simultaneously. Curated benchmarks such as HotpotQA [19] and MuSiQue [16] are multi-hop but small, hand-annotated, and rapidly saturated by capable models, whereas datasets such as SQuAD [11] and NewsQA [15] are large but do not require reasoning over different parts of the context.

To address this gap, we propose a fully automated pipeline for constructing large-scale, multi-hop QA datasets tailored to RL post-training. Our approach operates directly on raw document and proceeds in five stages: (i) extraction of structured relational triples grounded in named entities, (ii) NLI-based verification of each triple against its source passage, (iii) graph-based chaining of verified triples into multi-hop reasoning paths, (iv) heuristic chain filtering, and (v) synthesis of natural language questions from each chain. The result is a dataset of questions that are document-specific, multi-hop by construction, and associated with verifiable answer spans, satisfying all three criteria above.

Secondly, we instantiate this pipeline on the Institutional Books corpus [3] to generate QA pairs, and validate their quality by post-training Qwen2.5-7B-Instruct [18, 14] via Group Relative Policy Optimization (GRPO), Group Sequence Policy Optimization (GSPO) and REINFORCE Leave-One-Out (RLOO).

Finally, we conduct an empirical study across algorithms and hyperparameter configurations, analyzing the factors that drive performance gains and identifying key failure modes.

2 Background and Related Work

2.1 Reinforcement Learning for LLM Post-Training

RL has gained traction as a post-training paradigm for LLMs, particularly in tasks requiring faithful reasoning over external knowledge [8]. Unlike traditional fine-tuning, RL post-training leverages explicit reward signals to align model outputs with desired behaviors, such as verifiability and factual accuracy. This approach is especially valuable in knowledge-intensive applications, where models must rely on provided context rather than parametric knowledge acquired during pretraining.

The origins of RL for LLM post-training trace back to its success in mathematical reasoning, where rule-based reward functions provide unambiguous feedback. Early work demonstrated that RL could enhance logical consistency and problem-solving by rewarding intermediate steps or final answers. Extending this framework to open-domain question answering (QA) introduces new challenges, as rewards must be tied to document comprehension and multi-hop reasoning rather than superficial pattern matching.

Among RL methods, Proximal Policy Optimization (PPO) [17] has become a standard due to its stability and sample efficiency. PPO optimizes the policy by clipping the probability ratio between new and old policies, preventing destructive updates. In the context of LLMs, PPO has been effectively applied to tasks with delayed rewards, such as mathematical reasoning and factual QA.

2.2 Multi-Hop Question Answering

Early work in this area focused on curated datasets, such as HotpotQA [19] and MuSiQue [16], which were designed to evaluate a model’s ability to perform multi-step reasoning. In MuSiQue, multi-hop questions are created by linking connected single-hop questions, such that answering the resulting question requires combining the information needed to answer the underlying sub-questions [16]. These datasets typically include questions that cannot be answered from a single passage, necessitating the identification and combination of relevant information from multiple documents.

However, existing multi-hop QA datasets face several challenges. First, they are often small and hand-annotated, limiting their scalability for training large models. Second, they may be rapidly saturated by state-of-the-art LLMs, which can quickly memorize or exploit patterns in the data. Third, many datasets lack automated verification mechanisms, making it difficult to compute rewards or evaluate answers at scale. These limitations highlight the need for large-scale, automatically generated datasets that are both challenging and verifiable.

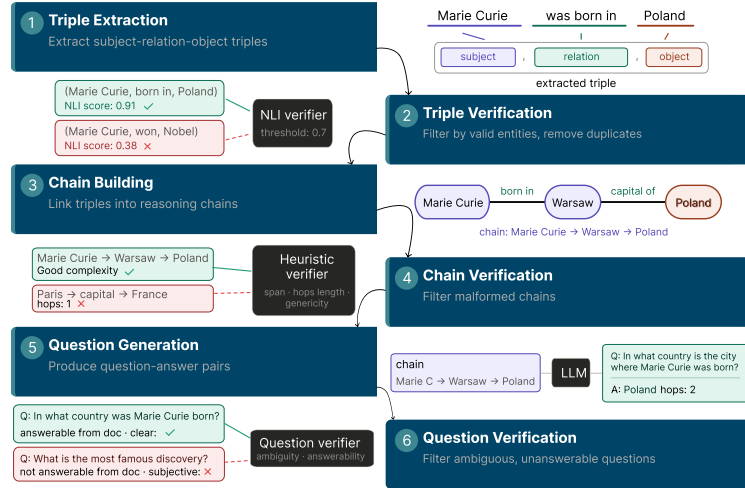


Figure 1: Overview of the six-stage pipeline used to generate multi-hop QA pairs from documents along with examples.

3 Approach

3.1 Dataset creation

To automatically create multi-hop QA pairs from the documents, we developed a modular pipeline that transforms raw documents into a set of verified questions and answers. Since our goal is to train models to reason over long but bounded contexts, each input document is first divided into contexts of a predefined length, which was set to 32,000 tokens in our experiments. The pipeline is then applied to each context individually, such that the generated questions can be answered using evidence from the context given to the LLM.

The pipeline consists of six stages, which can be seen in Figure 1. The modular structure allowed us to create several variant of the pipeline in order to later test their quality separately. Variants are defined by their name as: `variant name`.

3.1.1 Triple Extraction

For each context, the pipeline begins by extracting structured knowledge in the form of relation triples (subject, relation, object). We first split the contexts into logical groups of text called sections. Each section is processed one by one by a `TripleExtractor` in order to produce a list of triples. As these relationships are central to the dataset, we implemented two extraction techniques, giving us 2 variants:

- `variant nlp`: relies on a basic NLP pipeline: coreference resolution [10], OpenIE triple extraction [9], triple filtering with NER entities and deduplication.
- `variant hybrid`: augments the first one with LLM-based triples curation.

3.1.2 Triple Verification

Extracted triples are filtered with an NLI evidence check. For each candidate (subject, relation, object) triple, we convert the triple into a declarative hypothesis using a question-to-declarative-sentence model, here the subject and object define a relation question, and the extracted relation is used as the answer. The resulting hypothesis is paired with the triple’s evidence sentence and classified with a cross-encoder model for NLI, and only triples labeled as `entailment` are kept.

3.1.3 Chain building

The verified triples are represented as a directed graph, where entities are nodes and relations define edges between them. Candidate chains are constructed by following directed edges through the graph,

keeping only paths that do not revisit entities or reuse edges. We keep chains up to the configured maximum hop length, which is 2 in our experiments. The final entity in each path is used as the answer span, while intermediate entities are stored as bridge entities.

3.1.4 Chain verification

Chain verification is applied as a heuristic filter before question generation. First, we keep only chains with the required minimum number of hops and remove chains with empty answer spans. We then filter out shortcut chains, where the same answer can be reached from the start entity in fewer hops, since these do not require the intended multi-hop reasoning. Finally, we deduplicate near-identical chains, rank candidates using their NLI scores, and apply diversity filters that limit repeated answers, bridge entities, and evidence spans.

3.1.5 Question Generation

Once relation chains have been verified, we use a generative large language model text-to-text LLM to produce one question for each chain. For each question, the model is given the target answer, the ordered chain of facts and the corresponding evidence sentences. It is prompted to return a JSON object containing a question, an answer, and a short reasoning trace. The implementation also supports an optional refinement loop, where if heuristic checks identify issues such as invalid JSON, answer mismatch, the answer or a bridge entity appearing in the question, or a question that can be answered without using the full chain, the model is prompted to revise its previous output.

3.1.6 Quality Control

Quality Control serves as a final safeguard for the dataset creation. Its role is to prevent unanswerable, ambiguous and hallucinated QA to integrate the final dataset. While many options are available, we implemented answerability checks and basic knowledge checks. To check answerability, we give a text-to-text LLM the question along with the evidence and check it can answer it based on those. In the basic knowledge check, we want to ensure that questions are not answerable properly by an LLM if it is only given the question and no other input document.

3.2 LLM post-training validation

Using the generated dataset, we post-train Qwen2.5-7B-Instruct [14, 18], a finetuned model and improvement over the Qwen2 series of LLMs. We selected the 7.6 billion parameters version to allow both high quality outputs and a feasible post-training time. Larger models would also require larger datasets to see a significant improvement in their post-training performance.

Different frameworks exist to post-train LLMs, such as tinkr, Unsloth or verl. In this work, we use verl (Volcano Engine Reinforcement Learning), a highly scalable open-source library based on HybridFlow [13]. For simplicity and computational efficiency, we removed any KL loss during training.

3.2.1 Choice of policies

We tested three different policies, all critic-free for computational efficiency. Two are variants of the classical Proximal Policy Optimization (PPO) algorithm. As a recall, PPO optimizes a *clipped* surrogate objective:

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left\{ w_t(\theta) \hat{A}_t, \text{clip}(w_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right\} \right]$$

where $w_t(\theta) = \frac{\pi_{\theta}(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the importance ratio of token y_t and the advantage $\hat{A}_t$ is computed by a critic, that is, a different model. GRPO and GSPO retain this clipped objective, but eliminates the critic in slightly different fashions. Both these methods therefore reduce the memory footprint and improve computational efficiency by removing this value model. Furthermore, obtaining a reliable critic is challenging, especially in long-context. [20]

The last policy tested, RLOO, is a variant of the REINFORCE policy-gradient method. It constructs an unbiased advantage baseline using the Leave-One-Out technique.

GRPO (Group Relative Policy Optimization) Originally implemented by the DeepSeek team [12], GRPO bypasses the need for a critic by sampling G responses per prompt and computing the advantages relative to the responses' mean reward. That is, the advantage from GRPO becomes

$$\hat{A}_i = \frac{r(x, y_i) - \text{mean}(\{r(x, y_i)\}_{i=1}^G)}{\text{std}(\{r(x, y_i)\}_{i=1}^G)}$$

and GRPO then optimizes the following objective:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left\{ w_{i,t}(\theta) \hat{A}_i, \text{clip}(w_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right\} \right]$$

where all the tokens y_i share the advantage $\hat{A}_i$.

GSPO (Group Sequence Policy Optimization): developed initially for the Qwen models [20], GSPO is a modified version of GRPO aiming to fix its large variance and instability. Whereas GRPO (and PPO) typically optimize each token (token-level), GSPO aligns the reward and the optimization target by scoring at the sequence-level. The new objective becomes

$$\mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \min \left\{ s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right\} \right]$$

where the importance ratio $s_i(\theta)$ is based on sequence likelihood:

$$s_i(\theta) = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_{\theta}(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right)$$

The clipping is therefore applied to the entire responses instead of at the token-level.

RLOO (REINFORCE Leave-One-Out) [1]: RLOO constructs $\hat{A}_i$ via Leave-One-Out, i.e., using the $G - 1$ other samples: $\hat{A}_i = r_i - \frac{1}{G-1} \sum_{j \neq i} r_j$. The key idea is that the reward r_i is independent of the baseline, yielding an unbiased advantage baseline estimate.

3.2.2 Reward functions

In the context of LLM post-training, the reward is typically a scalar assigned to each complete model output, encoding whether the response is correct, well-formed, or aligned with some desired behaviour. In verl, the reward function is implemented as a Python callable registered per dataset, receiving the model's raw output string alongside the ground truth and any additional metadata stored at dataset creation time.

A central challenge in applying reinforcement learning to question answering is defining a reward signal that is both informative and automatically computable at scale. Unlike mathematical reasoning tasks where answers can be verified by symbolic execution, open-domain QA requires comparing natural language strings, which admits different level of correctness. We therefore decided to investigate several reward function in order to analyze the performance over different aspects.

For verifiable tasks such as mathematical reasoning, a binary reward is the standard choice and has shown strong empirical results. However, recent work [4] has suggested that strictly binary rewards can create a fundamental degeneracy for policy gradient methods. Motivated by this, we explored both binary and continuous reward formulations.

Baseline (binary): We started with a baseline reward function that only checks if model output is contained in ground truth. While this allowed us to get started with some results and direct our future runs, a such simple reward function suffer from the possibility of reward hacking, in which the model tends to output more and more copies of the initial context to get a valid reward.

Parameter	Description	Value
Ulysses sequence parallelism [5]	Splits sequence dimension across multiple GPUs during actor update & reference forward passes, reducing per-GPU attention memory by a factor equal to this value.	2
Rollout GPU memory utilization	Fraction of GPU memory allocated to vLLM instance during rollout.	0.4
Dynamic batch size	Automatically adjusts micro-batch sizes based on sequence lengths to maintain consistent GPU memory usage.	True
Max. token length per GPU	Token budget per GPU per forward or backward pass when dynamic batching is enabled.	131'072
Max. number of batched tokens	Max. number of tokens processed in a single prefill step across all active sequences.	131'072

Table 1: Key verl memory parameters used in our configurations.

String similarity (binary): As a slightly modified baseline, we use `token_sort_ratio` from the `rapidfuzz` [2] python library, which sorts tokens alphabetically before computing distance. This handles word-order variation and minor surface differences at a low computation cost. We transform the ratio into a binary output by validating an answer if ratio was more than a threshold (set to 0.6 here).

String similarity (continuous): We computed a continuous version of the string matching reward by outputting the similarity score from `rapidfuzz` divided by 100 to keep rewards clipped between 0 and 1. This score yields high rewards when the output is grammatically close to the ground truth, but not necessarily if the output is semantically close to the ground truth, encouraging the model to answer from the context and not from memory.

Reasoning (binary): Finally, we created a reward function whose goal is to reward the model only if it performs the reasoning steps needed to answer the question. To make reward computation tractable, we constrain the model to respond in a structured format using XML-like tags.

The structured output makes answer extraction fully deterministic via regular expression parsing, and inspectable for hop-level verification. Due to time constraints, we were not able to run any training using this reward function.

3.2.3 Post-training hyperparameters

We chose a batch size of 32 samples, as the datasets were of relatively small size, and trained for 20 epochs. Qwen2.5-7B-Instruct supports a maximum context window of 32'768 tokens, which we divided into a maximum prompt length of 32'000 tokens and a maximum response length of 768 tokens. The learning rate was fixed at 10^{-6} . The rollout number — the number of answers generated by the model at each samples — was set to 4. This is lower than typical values in the literature, but was necessary to keep training time tractable given the project’s short timeline.

A major difficulty when training with long context ($\geq 32k$ tokens) is managing GPU memory: the goal is to maximize throughput and speed without triggering an Out Of Memory (OOM) error. The verl framework contains a number of parameters to help finding this balance. We briefly discuss the most relevant ones in Table 1, and provide the full configuration for each policy algorithm in Appendix B.

4 Results

4.1 Experimental Setup

All experiments were conducted on the Clariden cluster at the Swiss National Supercomputing Centre (CSCS), a high-performance computing system equipped with NVIDIA GH200 GPUs. Both the

Method	Verified triples	Generated QAs			Total QAs	Multi-hop QAs	Multi-hop (%)
		1-hop	2-hop	3-hop			
Engineered OpenIE	5,465	1,605	221	52	1,878	273	14.5
Hybrid OpenIE+LLM	8,931	2,465	767	641	3,873	1,408	36.4

Table 2: Comparison of verified triples and QA pairs for the two extraction methods. Verified triples are triples that passed NLI verification. Multi-hop QAs include 2 and 3-hop questions.

dataset creation pipeline and the RL post-training jobs were allocated 4 NVIDIA GH200 GPUs per run, with job scheduling managed via SLURM.

Dataset creation was carried out in separate stages due to the long runtime of the pipeline. In a first run, triples were extracted and verified from the document corpus. In a subsequent run, the verified triples were used to construct relation chains and generate the corresponding questions. RL post-training was also computationally demanding: each complete training run required approximately 24 hours on the 4-GPU allocation. Due to a maximum node allocation of 12 hours, two runs were needed by experiment, and a checkpoint was made every 12 iterations with a restart set at the most recent checkpoint.

Managing access, scheduling jobs on the right environment and debugging such large scale experiment were as interesting as challenging.

4.2 Dataset creation

Running our automated dataset creation pipeline on Clariden, we successfully generated one dataset per pipeline variant using one book from [3]. Both datasets contains around 500 2-hop questions and display a significant level of difficulty. You can find some examples below:

Question: Which valley is the destination of the southeastward drainage
from the elevated western region of Rhode Island?
Answer: Connecticut Valley

We evaluated both extraction methods on a set of ten documents, using the same downstream verification pipeline. The resulting numbers of verified triples and generated QA can be seen in Table 2.

The hybrid extractor produces more triples that are verified by NLI, as well as more QA pairs that remain after the chain verification stage. The higher proportion of multi-hop QAs also suggests that the graph created from the hybrid triples may be more connected, potentially due to improved entity canonicalization. However, further analysis would be required to determine whether this is the cause of the difference.

4.3 RL post-training

Figure 2 summarizes our key results. Each line corresponds to a dataset, and each column represents a different reward function. The graphs shown represent the mean scores on a held-out dataset (20% of the full dataset). Note that in the case of the binary reward function, these scores are equivalent to the percentage of right answers.

Overall, we can see that the LLM is improving at answering the questions from our generated datasets. We observe that the binary reward function induces better model improvement on both dataset variants but also displays stronger differences between each algorithms. However, as we were unable to run multiple trainings per model per dataset, we lack the statistical results to make claims about algorithms’ differences. Secondly, we see that the continuous reward function shows a higher validation score overall. This is not surprising, as the mean value here is not binary, therefore even wrong answers will get partial credits. However, we see little to no progress using this reward function. This could be explained by the lack of a strong reward signal: the model receives a non-zero value most of the time and therefore has difficulty learning the actual task.

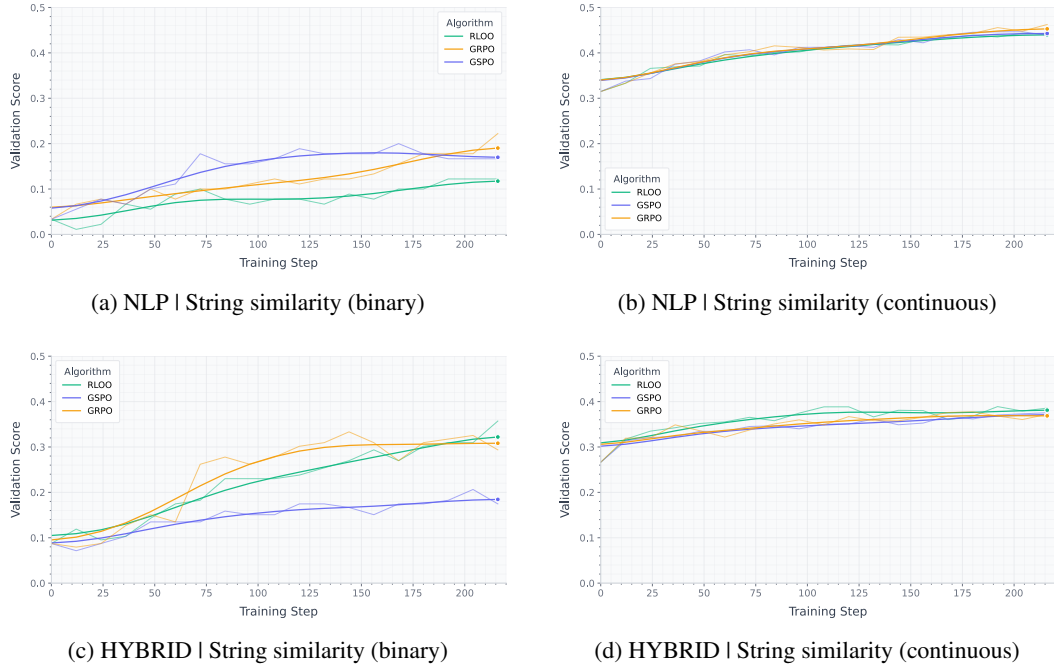


Figure 2: Validation score by dataset and reward function type

We also observe that the hybrid dataset is partially answerable by the model without any training (initial score ~ 0.1), contrary to the NLP variant (initial score ~ 0.05). When using the continuous reward function, this difference is inverted: the NLP version has higher initial score (~ 0.35) than the hybrid one (~ 0.3). Further research into the quality of the generated questions should be done to explain those differences.

Difficulties in RL post-training As discussed above, optimizing GPU memory was a major challenge during post-training. Our initial configuration required approximately 450 seconds per iterations. After additional tuning, we reduced this to roughly 300 seconds per iterations. While not fully optimal, time constraints prevented further optimization and all experiments were run under this configuration.

A second obstacle we encountered was cluster scheduling. Running multiple algorithms with different reward functions resulted in a high volume of jobs, which progressively reduced our Slurm priority. As a consequence, we faced delays of several days and were not able to run as many experiments as originally planned; particularly, we were not able to run multiple iterations using the same configuration to report standard deviations on our results.

5 Conclusion

Upgrading LLM with the ability to use a long context to better answer question is a tremendous but necessary task. LLM post-training on multi-hop QA datasets have been shown to improve models in this direction but some key limitations slow down the process.

To overcome those limitation, we built a dynamically adjustable pipeline for automatically generating multi-hop document-grounded questions from a large corpus of text. We validated the quality of the questions by post-training Qwen2.5, experimenting several RL hyper-parameters or reward functions, all showing that the model is able to learn how to provide valid answers from our generated dataset.

6 Acknowledgments

We are grateful to Eduard Ďurech and Valentina Pyatkin from the Swiss AI team for giving us the opportunity to work on this project and for their time and helpful advices. We also thank Juan Garcia Giraldo for his help with optimizing vERL, saving us hours of computing.

References

- [1] Arash Ahmadian, Chris Cremer, Matthias Gall , Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet  st n, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 20a24.
- [2] Max Bachmann. Rapidfuzz: Rapid fuzzy string matching in python using various string metrics, 2026. GitHub repository.
- [3] Matteo Cargnelutti, Catherine Brobston, John Hess, Jack Cushman, Kristi Mukk, Aristana Scourtas, Kyle Courtney, Greg Leppert, Amanda Watson, Martha Whitehead, and Jonathan Zittrain. Institutional books 1.0: A 242b token dataset from harvard library’s collections, refined for accuracy and usability, 2025.
- [4] Marc Dymetman. Binary rewards and reinforcement learning: Fundamental challenges, 2026.
- [5] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepseek ulysses: System optimizations for enabling training of extreme long sequence transformer models, 2023.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K ttler, Mike Lewis, Wen tau Yih, Tim Rockt schel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [7] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhua Chen. Long-context llms struggle with long in-context learning, 2024.
- [8] Keliang Liu, Dingkan Yang, Ziyun Qian, Weijie Yin, Yuchi Wang, Hongsheng Li, Jun Liu, Peng Zhai, Yang Liu, and Lihua Zhang. Reinforcement learning meets large language models: A survey of advancements and applications across the llm lifecycle, 2025.
- [9] Pai Liu, Wenyang Gao, Wenjie Dong, Lin Ai, Ziwei Gong, Songfang Huang, Zongsheng Li, Ehsan Hoque, Julia Hirschberg, and Yue Zhang. A survey on open information extraction from rule-based model to large language model, 2024.
- [10] Shon Otmazgin, Arie Cattan, and Yoav Goldberg. F-coref: Fast, accurate and easy to use coreference resolution, 2022.
- [11] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text, 2016.
- [12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [13] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- [14] Qwen Team. Qwen2.5: A party of foundation models, September 2024.
- [15] Adam Trischler, Tong Wang, Xingdi Yuan, Justin Harris, Alessandro Sordoni, Philip Bachman, and Kaheer Suleman. NewsQA: A machine comprehension dataset. In Phil Blunsom, Antoine Bordes, Kyunghyun Cho, Shay Cohen, Chris Dyer, Edward Grefenstette, Karl Moritz Hermann, Laura Rimell, Jason Weston, and Scott Yih, editors, *Proceedings of the 2nd Workshop on Representation Learning for NLP*, pages 191–200, Vancouver, Canada, August 2017. Association for Computational Linguistics.

- [16] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- [17] Zhichao Wang, Kiran Ramnath, Bin Bi, Shiva Kumar Pentyala, Sougata Chaudhuri, Shubham Mehrotra, Zixu, Zhu, Xiang-Bo Mao, Sitaram Asur, Na, and Cheng. Reinforcement learning for llm post-training: A survey, 2026.
- [18] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- [19] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018.
- [20] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization, 2025.
- [21] Fengbin Zhu, Wenqiang Lei, Chao Wang, Jianming Zheng, Soujanya Poria, and Tat-Seng Chua. Retrieving and reading: A comprehensive survey on open-domain question answering, 2021.

7 Appendix

A RL - Post-training mean rewards

Figure 3 displays the mean reward given to the critic during training.

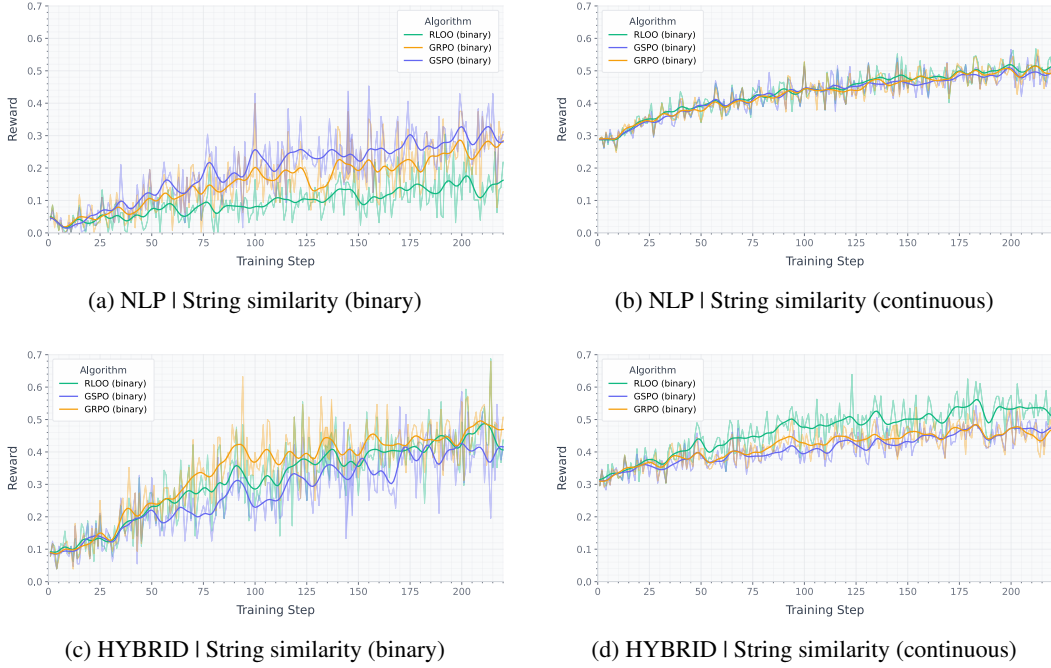


Figure 3: Mean reward by dataset and reward function type

B vERL configuration

B.1 GRPO

Listing 1: Full GRPO configuration.

```
# Algorithm
algorithm.adv_estimator=grpo
algorithm.use_kl_in_reward=False

# Data
data.train_files="/path/to/train/parquet"
data.val_files="/path/to/test/parquet"
data.train_batch_size=32
data.max_prompt_length=32000
data.max_response_length=768
data.filter_overlong_prompts=True
data.truncation='error'

# Model
actor_rollout_ref.model.path="Qwen/Qwen2.5-7B-Instruct"
actor_rollout_ref.model.use_remove_padding=True
actor_rollout_ref.model.enable_gradient_checkpointing=True
actor_rollout_ref.model.enable_activation_offload=True
actor_rollout_ref.model.use_fused_kernels=True

# Actor
actor_rollout_ref.actor.strategy=fsdp2
actor_rollout_ref.actor.use_kl_loss=False
actor_rollout_ref.actor.optim.lr=1e-6
actor_rollout_ref.actor.ppo_mini_batch_size=32
actor_rollout_ref.actor.use_dynamic_bsz=True
actor_rollout_ref.actor.ppo_max_token_len_per_gpu=131072
actor_rollout_ref.actor.ulysses_sequence_parallel_size=2
actor_rollout_ref.actor.fsdps_config.param_offload=False
actor_rollout_ref.actor.fsdps_config.optimizer_offload=False
actor_rollout_ref.actor.fsdps_config.forward_prefetch=True

# Rollout
actor_rollout_ref.rollout.name=vllm
actor_rollout_ref.rollout.disable_log_stats=False
actor_rollout_ref.rollout.tensor_model_parallel_size=1
actor_rollout_ref.rollout.gpu_memory_utilization=0.4
actor_rollout_ref.rollout.n=4
actor_rollout_ref.rollout.log_prob_use_dynamic_bsz=True
actor_rollout_ref.rollout.log_prob_max_token_len_per_gpu=131072
actor_rollout_ref.rollout.max_num_batched_tokens=131072
actor_rollout_ref.rollout.max_num_seqs=128
actor_rollout_ref.rollout.multi_stage_wake_up=True
actor_rollout_ref.rollout.enable_chunked_prefill=False
actor_rollout_ref.rollout.enforce_eager=False
actor_rollout_ref.rollout.free_cache_engine=True

# Reference
actor_rollout_ref.ref.log_prob_use_dynamic_bsz=True
actor_rollout_ref.ref.log_prob_max_token_len_per_gpu=131072
actor_rollout_ref.ref.ulysses_sequence_parallel_size=2
actor_rollout_ref.ref.fsdps_config.param_offload=True

# Reward
custom_reward_function.path="/path/to/reward/python/file"
custom_reward_function.name=similarity

# Trainer
trainer.resume_mode=auto
trainer.balance_batch=True
trainer.logger='["console", "wandb"]'
trainer.project_name=qa_rl
trainer.experiment_name=qwen2.5_vl_7b_grpo_hybrid_similarity
trainer.n_gpus_per_node=4
trainer.nnodes=1
trainer.val_before_train=True
```

```
trainer.save_freq=12
trainer.test_freq=12
trainer.total_epochs=20
trainer.default_local_dir="/path/to/checkpoints/directory/"

# Ray
ray_kwargs.ray_init.num_cpus=32
```

B.2 GSPO

We mention only the parameters that differ from the GRPO configuration.

Listing 2: Partial GSPO configuration.

```
# Actor
actor_rollout_ref.actor.policy_loss.loss_mode=gspo
actor_rollout_ref.actor.loss_agg_mode=seq-mean-token-mean
actor_rollout_ref.actor.clip_ratio_low=3e-4
actor_rollout_ref.actor.clip_ratio_high=4e-4
actor_rollout_ref.actor.clip_ratio_c=10.0
```

B.3 RLOO

We mention only the parameters that differ from the GRPO configuration.

Listing 3: Partial RLOO configuration.

```
# Algorithm
algorithm.adv_estimator=rloo
```